Deployment Strategy Analysis

1. Overview

To deploy the OBE Tracking System and make it accessible online, two cloud deployment approaches were evaluated:

• Deployment using Render
• Deployment using Amazon Web Services (AWS)

The objective of this analysis is to understand the infrastructure requirements, deployment process, time involved, and cost implications associated with each approach. This document is intended to provide **decision support** by outlining the technical considerations and trade-offs of each option.

---

2. Deployment Plan Using Render

Render is a cloud platform that simplifies application deployment by integrating directly with Git repositories and automating build and deployment processes.

Proposed Architecture

React Frontend

↓

Render Web Service

↓

Django Backend

↓

Render PostgreSQL Database

Deployment Workflow

1. Push project code to GitHub repository.
2. Connect the repository to Render.
3. Configure build and start commands.
4. Render automatically builds and deploys the application.
5. The platform provides a public URL to access the system.

Example deployment URL

https://project-name.onrender.com

Key Characteristics

- Automatic deployment from Git repositories
- Built-in HTTPS and public domain generation
- Minimal infrastructure configuration
- Free tier available for small projects

Limitations of Free Tier

- The web service enters sleep mode after approximately 15 minutes of inactivity.
- First request after inactivity may experience a cold start delay.
- Limited database storage in the free tier.
- Less infrastructure-level customization compared to full cloud platforms.

Overall Time Breakdown (Render)

Before deployment, the project requires system validation and repository preparation.

Task	**Time**
System validation (runtime and logical error fixing)	2–3 days
Output verification and stabilization	included in validation phase
Git repository cleanup (removing unused branches, merging to main)	1–2 hours
Render deployment setup	10–20 minutes
Cloud testing after deployment	30–45 minutes

Total realistic timeline

Minimum: **2 days**
Maximum: **3–4 days**

Most of the time is spent **validating outputs and stabilizing the system**, not in the deployment process itself.

Overall Cost Breakdown (Render)

Typical cost estimates for deployment using Render.

Component	Cost
Web service	Free tier available
PostgreSQL database	Free tier available
Storage and bandwidth	Included within free limits

Estimated total monthly cost

Minimum: **₹0**
Maximum (if paid plans are used): **₹600 – ₹1200 per month**

---

3. Deployment Plan Using AWS

AWS provides a flexible cloud infrastructure where application components are deployed and managed individually.

The architecture considered uses commonly adopted deployment components.

Proposed Architecture

React Frontend

↓

Nginx Reverse Proxy

↓

Gunicorn Application Server

↓

Django Backend

↓

PostgreSQL Database

Infrastructure services used:

• Amazon EC2 for application hosting
• Amazon RDS for PostgreSQL database management
• Amazon Elastic Block Store for server storage

Deployment Workflow

1. Launch EC2 instance and configure the operating system.
2. Install dependencies such as Python, Git, and Nginx.
3. Clone project repository from GitHub.
4. Configure Gunicorn to run the Django application.
5. Configure Nginx as a reverse proxy server.
6. Create and configure PostgreSQL database using RDS.
7. Run database migrations and start application services.

Key Characteristics

• Full control over infrastructure configuration
• No automatic service sleep behavior
• Industry-standard production architecture
• High scalability and flexibility

Limitations

• Higher setup complexity
• Requires manual server configuration and maintenance
• Higher operational cost compared to simplified platforms

Overall Time Breakdown (AWS)

The system preparation steps remain the same as they are independent of the deployment platform.

Task	**Time**
System validation (runtime and logical error fixing)	2–3 days
Output verification and stabilization	included in validation phase

Task	Time
Git repository cleanup (removing unused branches, merging to main)	1–2 hours
AWS infrastructure setup (EC2 + RDS + networking)	30–40 minutes
Server configuration (Nginx + Gunicorn)	30–40 minutes
Database configuration and migrations	15–20 minutes
Backend and frontend deployment	20–30 minutes
Cloud testing after deployment	30–60 minutes

Total realistic timeline

Minimum: **2 days**
Maximum: **3–4 days**

The majority of the timeline again comes from **system validation and stabilization**, while the cloud deployment process itself typically takes **1.5–2 hours**.

Overall Cost Breakdown (AWS)

Estimated monthly infrastructure cost for the proposed architecture.

Component	Cost
EC2 instance	₹600 – ₹850
RDS PostgreSQL database	₹1000 – ₹1700
EBS storage	₹100 – ₹300
Network transfer	₹50 – ₹200

Estimated total monthly cost

Minimum: **₹1750 per month**
Maximum: **₹5000 – ₹8000 per month** depending on instance sizes and resource usage.

If free-tier resources apply, the cost may reduce significantly for small deployments.

---

4. Comparison of Deployment Approaches

Factor	Render	AWS
Deployment complexity	Low	Moderate
Deployment time	10–20 minutes	1.5–2 hours
Infrastructure control	Limited	Full control
Cost	Lower	Higher
Cold start behavior	Possible	None
Scalability	Moderate	High
Learning value	Moderate	High
Operational management	Platform managed	User managed

---

5. Trade-Off Analysis

Deployment platforms present different trade-offs depending on project goals and constraints.

Simplicity vs Infrastructure Control
Render simplifies deployment through automated build and deployment pipelines, reducing the need for manual configuration. AWS provides deeper infrastructure control but requires configuration of multiple components such as servers, proxies, and databases.

Cost vs Flexibility
Render's free tier enables low-cost deployment suitable for small projects. AWS infrastructure introduces higher operational cost but provides flexible resource scaling and customization.

Deployment Speed vs Customization
Render allows rapid deployments directly from version control systems. AWS deployments require additional configuration but enable customization of networking, security, and server environments.

Operational Responsibility
Render manages most infrastructure components automatically. AWS deployments require manual management of services, updates, and server configurations.

---

6. Deployment Risks and Mitigation

During cloud deployment, several technical risks may affect system availability, performance, or data reliability. Identifying these risks helps ensure stable operation of the OBE Tracking System after deployment.

Render Deployment Risks

Cold Start Delay
In the free tier of Render, services automatically enter sleep mode after inactivity. When a user accesses the system after this period, the application must restart, which can cause a delay of several seconds.

Mitigation
For demonstration or frequent usage, periodic access to the system can keep the service active. Paid plans also remove or reduce this limitation.

Limited Database Resources
Free database plans provide limited storage and memory resources, which may affect performance when handling larger datasets or reports.

Mitigation
Database usage can be optimized by limiting unnecessary data storage and cleaning temporary files. If required, the database plan can be upgraded.

Platform Dependency
Render abstracts infrastructure management. If the platform experiences downtime, developers have limited control over the underlying infrastructure.

Mitigation
Maintaining code and database backups enables migration to other cloud platforms if required.

AWS Deployment Risks

Configuration Errors
Manual configuration of infrastructure on Amazon Web Services may introduce errors in server setup, firewall rules, or service configuration.

Mitigation
Following structured deployment steps and testing each configuration stage helps reduce configuration-related issues.

Server Management Overhead
When deploying on AWS, the development team is responsible for maintaining servers, updating software, and monitoring system performance.

Mitigation
Basic server monitoring tools and scheduled updates can help maintain system stability.

Infrastructure Cost Variability
AWS operates on a pay-as-you-use model, meaning costs can increase if additional resources are used or if services remain active for long durations.

Mitigation
Monitoring usage through AWS billing dashboards and limiting resource allocation can help control costs.

Common Risks Across Both Platforms

Application Runtime Errors
Logical errors or runtime exceptions in the application may appear only after deployment under real usage conditions.

Mitigation
The system is currently undergoing validation and stabilization to ensure accurate attainment calculations, report generation, and data uploads before deployment.

Repository and Version Control Issues
Multiple unused branches or inconsistent code versions can cause deployment failures.

Mitigation
Before deployment, the Git repository will be cleaned by removing unused branches and merging validated code into the main branch.

Data Integrity During Uploads
The system supports Excel and PDF uploads for academic data. Improper file formats or incorrect data structures may affect processing.

Mitigation
Input validation mechanisms and structured file templates can reduce data-related errors.
